

Assignment 9: Neural Nets

AUTHOR

Talha Gulzar

PUBLISHED

November 23, 2024

Context

Let's take a look at the data located in UniversalBank.csv. Each row represents a customer at small but rapidly growing bank. The columns measure all sorts of customer characteristics, ranging from their demographic information (e.g., Age, Family) to whether they currently have various accounts open with the bank (e.g., Securities Account, CD Account). For a complete description of the fields, consult the data page on Canvas.

The bank is aggressively trying to convert customers from depositors into borrowers through its personal loan program. The column **Personal Loan** shows whether each customer responded to a direct marketing campaign related to this program. The marketing team is now trying to understand what types of customers responds to new personal loans marketing. If they can establish reasonably strong predictive power, they will deploy the model more widely across their customer base to identify promising leads and a more nuanced target market.

Task 1: Predicting Mortgage Amount

Question 1

Do the following:

- Import the data as "bank"
- Remove any predictor that might be inappropriate for this activity.
- Set the target to a factor.

```
library(nnet)
library(caret)
```

Loading required package: ggplot2

Loading required package: lattice

```
library(NeuralNetTools)
```

Warning: package 'NeuralNetTools' was built under R version 4.4.2

```
source("BabsonAnalytics.R")

# import data
bank = read.csv('UniversalBank.csv')

bank$ID = NULL
bank$ZIP.Code = NULL

factor_columns = c("Family", "Education", "Securities.Account", "CD.Account", "Online", "CreditCard",
bank[factor_columns] = lapply(bank[factor_columns], factor)
```

Question 2

- Given the fact that most activation functions are nonlinear, standardizing is an important step towards making sure no one variable swamps all of the others.

Standardize all of the numeric inputs for this data set (e.g., using `preProcess` from the `caret` package).

Note: During class, we normalized our variable. This time, we want to standardize! We have done this in the past!

To verify that this process has gone well, enter the maximum value of the standardized version of the Income variable below. Round your answer to two decimal places.

Answer: 3.26

```
standardizer <- preProcess(bank, c("center", "scale"))
bank <- predict(standardizer, bank)

income_maxout <- max(bank$Income)
income_maxout
```

```
[1] 3.263385
```

Question 3

Write a short sentence putting describing what the quantity you found in the previous question (Q3) means.

Answer: The value suggests that the data is standardized, and the value now shows that the maximum value of income deviates by 3.26 from the mean of income.

Question 4

- Set a seed of 28 and partition a training set with 70% of the data.

```
set.seed(28)
N = nrow(bank)
trainingSize = round(N*0.7)
trainingCases = sample(N, trainingSize)
train = bank[trainingCases,]
test = bank[-trainingCases,]
```

Question 5

Build a neural net model with a hidden layer of size 4, and plot it using the plotnet function from the NeuralNetTools package. Notice how bad this default plotting is. Really, really subpar. Now plot with the fix (originally posted here: <https://github.com/fawda123/NeuralNetTools/issues/20>).

```
par(mar = numeric(4)) plotnet(model, pad_x = .5)
```

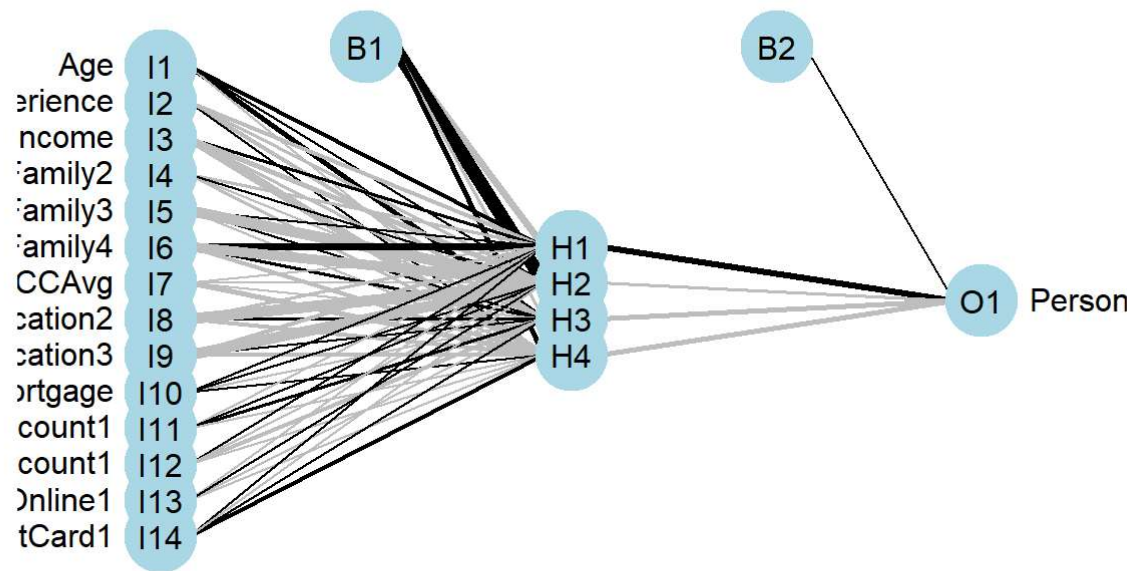
What was the difference between the original and the fixed versions?

Answer: The fixed version has a better display of variables which are easier to distinguish, while in the original, the predictors are crammed together and are hard to determine the exact name of predictors as some names are not completely displayed. Besides this, the weighted lines are also better distinguishable in the fixed graph rather than the original one, thus comparatively easier to determine the magnitude of the weight each predictor has. The lines in the original are interlaced so it's confusing to distinguish the actual weights that are impacting the actual target.

```
# Run the Neural net model
model <- nnet(Personal.Loan ~ ., data=train, size = 4)
```

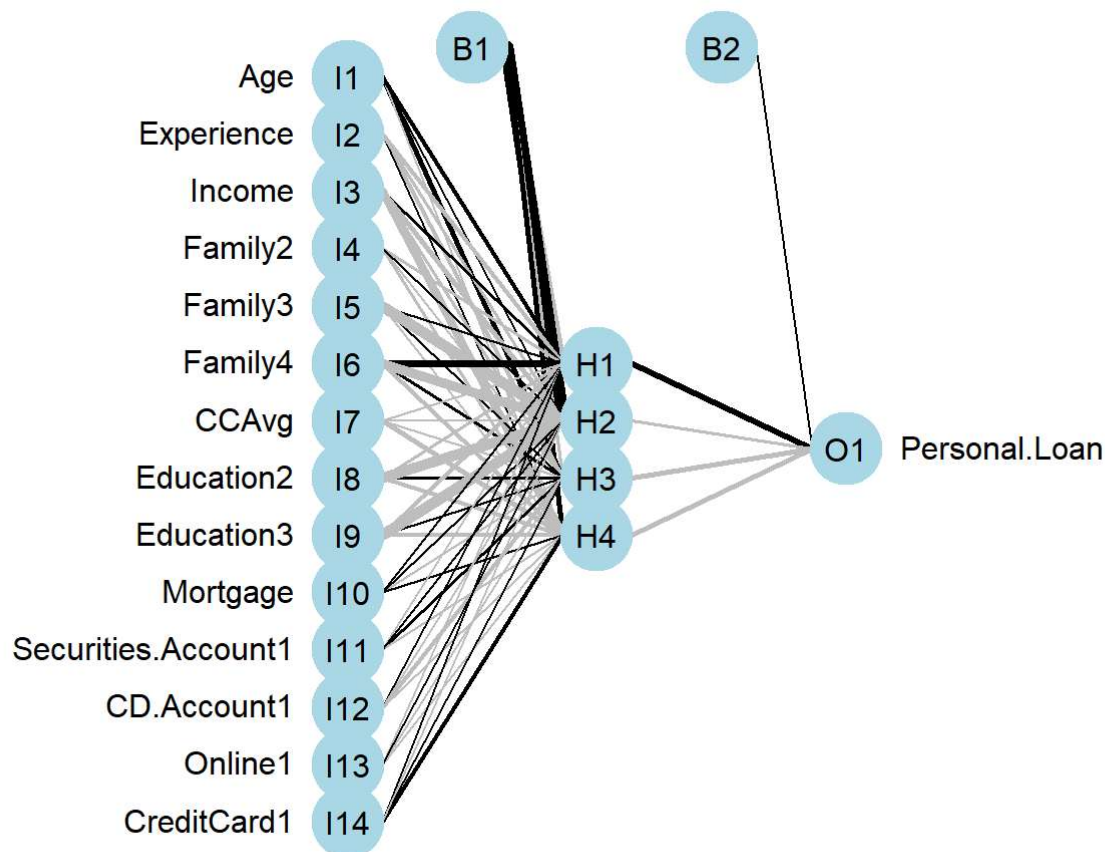
```
# weights: 65
initial value 2333.558111
iter 10 value 624.206800
iter 20 value 304.457534
iter 30 value 249.476305
iter 40 value 231.503473
iter 50 value 213.155326
iter 60 value 203.253336
iter 70 value 180.869262
iter 80 value 139.183119
iter 90 value 116.566725
iter 100 value 109.622363
final value 109.622363
stopped after 100 iterations
```

```
plotnet(model)
```



#Github Code

```
par(mar = numeric(4))
plotnet(model, pad_x = .5)
```



Question 6

What can we learn about what types of customers accept personal loan offers by visualizing examining the neural net you've created? (i.e., what did we say in class from looking at this visual?)

Answer: The initial layer comprises of predictors, and the lines that go to the hidden layer tells us information about the weight of the variable. The final output is reached by going through the hidden layers, which are 4 in this case. By checking the visual we can focus on thick black lines leading from the predictor to be classifiers of predictors which can have a important positive impact on the target variable. So in this case, Age and the fourth category in Family has a strong positive influence as it has a thick black line, thus the weight assigned to them is positive. While income, CC Avg and education have a strong negative correlation as they both have thick gray lines, thus the weight assigned to them is negative.

Question 7

Take a look at your confusion matrix. Also, calculate the error rate and benchmark error rate. How many predictions did you get correct from the model? What is the error rate associated with your model? Is this a useful model?

Answer: 1463 predictions were made correctly. So when considering the error of the model which is 0.02466, it is lower than the benchmark error rate which is 0.0893, thus suggesting that the model is useful.

```
# make predictions
pred = predict(model, test, type="class")

# See confusion matrix
table(pred, test$Personal.Loan)
```

```
pred    0    1
  0 1344   15
  1   22  119
```

```
correct_predictions<- 1344+119

# calculate error rate
error_rate = sum(pred != test$Personal.Loan)/nrow(test)
error_rate
```

```
[1] 0.02466667
```

```
#calculate benchmark error rate
error_bench = benchmarkErrorRate(train$Personal.Loan, test$Personal.Loan)
error_bench
```

```
[1] 0.08933333
```

Question 8

One could make the argument that we care very much about predicting the True observations correctly in this model. What is sensitivity of this model? Round your answer to two decimal places, e.g., 0.12

Also, how does this rate - essentially an accuracy rate - compare with the error rate you obtained previously? What is the issue with this if predicting True's accurately was really important to you in this context?

Answer: The sensitivity of the model is 88.80%. This value suggests that this 88.80% of observed trues were predicted correctly by the model. So considering the importance of true being predicted correctly, there would be 88.80% of these trues that would be predicted correctly. While the sensitivity rate is reasonably high, the overall error rate from Question 7 might not fully reflect the model's performance on True cases as it also focuses on the false cases which were actually not false too .

```
sensitivity <- sum(pred == 1 & test$Personal.Loan == 1)/sum(test$Personal.Loan == 1)
sensitivity
```

[1] 0.8880597

Question 9

What happens if you don't standardize? Turn off standardization and re-run your neural network(i.e., Just use hashtags to comment out a couple lines in your code and re-run everything). You will likely get an error if you try to compute the error rate, so just look at the misclassification table instead. What do you see? What you do think happened here?

After you answer this question, just make sure to un-comment your lines of code before knitting.

Answer:While the model is not very excellent at predicting positive trues, it is good at predicting false positives. This might be occurring because the data is not normalized, which would cause some variables to have high values and be given greater weight while casting other variables in a shadow.